

Module 2

Detailed *Notes*

Chapter 4 — Statements

Topics Covered

- True and False in C
- Selection Statements — if, Nested ifs, if-else-if Ladder, ? Alternative, switch
- Iteration Statements — for Loop (and Variations), while Loop, do-while Loop
- Jump Statements — return, goto, break, continue, exit()
- Expression Statements
- Block Statements

CHAPTER 4 — STATEMENTS

In the most general sense, a statement is a part of your program that can be executed — it specifies an action. C categorizes statements into the following groups:

Category	Statements
Selection	if, switch (also called conditional statements)
Iteration	while, for, do-while (also called loop statements)
Jump	break, continue, goto, return
Label	case, default (used with switch), label (used with goto)
Expression	A valid expression followed by a semicolon
Block	A block of code enclosed in { } (compound statements)

True and False in C

Many C statements rely upon a conditional expression that determines what course of action is to be taken. A conditional expression evaluates to either a true or false value.

Value	Meaning in C
Any nonzero value (including negatives)	TRUE
0 (zero)	FALSE

This approach to true and false allows a wide range of routines to be coded extremely efficiently.

SELECTION STATEMENTS

C supports two selection statements: if and switch. In addition, the ? operator is an alternative to if in certain circumstances.

The if Statement

The general form of the if statement is:

```
if (expression) statement;  
else statement;
```

Where a statement may consist of a single statement, a block of statements, or nothing (empty statement). The else clause is optional.

- If expression evaluates to true (anything other than 0), the statement or block forming the target of if is executed
- Otherwise, the statement or block that is the target of else executes (if it exists)
- Only the code associated with if OR the code associated with else executes — never both

- The conditional expression controlling if must produce a scalar result: integer, character, pointer, or floating-point type
- In C99, `_Bool` is also a scalar type and may be used in an if expression
- It is rare to use a floating-point number to control a conditional statement — it slows execution considerably

NOTE *An integer or character operation requires relatively few instructions; a floating-point operation takes several instructions.*

Example — Magic Number Program #1 (basic if):

```
/* Magic number program #1 */
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int magic;    /* magic number */
    int guess;    /* user's guess */
    magic = rand();
    printf("Guess the magic number: ");
    scanf("%d", &guess);
    if(guess == magic) printf("** Right
**");
    return 0;
}
```

Example — Magic Number Program #2 (if-else):

```
if(guess == magic) printf("** Right **");  
else printf("Wrong");
```

Nested ifs

A nested if is an if that is the target of another if or else. In a nested if, an else statement always refers to the nearest if statement that is within the same block and not already associated with an else.

```
if(i)  
{  
    if(j) dosomething1();  
    if(k) dosomething2();    /* this if */  
    else dosomething3();    /* is associated  
with this else */  
}  
else dosomething4();        /* associated  
with if(i) */
```

- The final else above is associated with if(i), not if(j), because it is not in the same block as if(j)
- The inner else is associated with if(k) — the nearest if
- C89 specifies at least 15 levels of nesting must be supported
- C99 raises this limit to 127
- Excessive nesting beyond a few levels is seldom necessary and can quickly confuse the meaning of an algorithm

Example — Magic Number Program #3 (nested if providing feedback):

```
if(guess == magic) {
    printf("** Right **");
    printf(" %d is the magic number\n",
magic);
}
else {
    printf("Wrong, ");
    if(guess > magic) printf("too high\n");
/* nested if */
    else printf("too low\n");
}
```

The if-else-if Ladder

A common construct is the if-else-if ladder (also called the if-else-if staircase). Its general form is:

```
if (expression)
    statement;
else if (expression)
    statement;
else if (expression)
    statement;
.
.
.
else
    statement;
```

- Conditions are evaluated from top to bottom

- As soon as a true condition is found, the statement associated with it is executed and the rest of the ladder is bypassed
- If none of the conditions are true, the final else executes
- If the final else is not present, no action takes place when all conditions are false

Example — Magic Number Program #4 (if-else-if ladder):

```
if(guess == magic) {  
    printf("** Right ** ");  
    printf("%d is the magic number",  
magic);  
}  
else if(guess > magic)  
    printf("Wrong, too high");  
else  
    printf("Wrong, too low");
```

The ? Alternative

The ? operator (ternary operator) can replace if-else statements of the form:

```
if (condition) var = expression;  
else var = expression;
```

The ? requires three operands (hence ternary). Its general form is:

```
Exp1 ? Exp2 : Exp3
```

- Exp1 is evaluated first
- If Exp1 is true, Exp2 is evaluated and becomes the value of the entire ? expression
- If Exp1 is false, Exp3 is evaluated and its value becomes the value of the expression

```
x = 10;  
y = x>9 ? 100 : 200;    /* y = 100 */  
  
/* Equivalent if-else: */  
if(x>9) y = 100;  
else y = 200;
```

The ? operator can also be used with function calls. For example:

```
t ? f1(t) + f2() : printf("zero entered.");  
/* Calls f1() and f2() if t is nonzero,  
else calls printf() */
```

NOTE *It is permissible for a compiler to rearrange the order of evaluation of an expression to optimize object code. This could cause calls in a ? expression to execute in an unexpected sequence.*

Example — Magic Number Program #5 (? operator):

```
if(guess == magic) {  
    printf("** Right ** ");  
}
```

```
    printf("%d is the magic number",
magic);
}
else
    guess > magic ? printf("High") :
printf("Low");
```

The Conditional Expression

Any valid expression can control the `if` or the `?` operator — not just expressions involving relational and logical operators (unlike BASIC or Pascal). The expression must simply evaluate to a true (nonzero) or false (zero) value.

Example — using a variable directly to avoid divide-by-zero:

```
if(b) printf("%d\n", a/b);
else printf("Cannot divide by zero.\n");

/* Writing if(b != 0) is redundant,
potentially inefficient,
and considered bad style. The value of b
alone is sufficient. */
```

The switch Statement

C has a built-in multiple-branch selection statement called `switch`, which successively tests the value of an expression against a list of integer or character constants. When a match is found, the statements associated with that constant are executed.

General form:

```
switch (expression) {  
    case constant1:  
        statement sequence  
        break;  
    case constant2:  
        statement sequence  
        break;  
    case constant3:  
        statement sequence  
        break;  
    .  
    .  
    default:  
        statement sequence  
}
```

- The expression must evaluate to an integer type — character or integer values are allowed; floating-point expressions are NOT allowed
- The value of expression is tested against each case constant in order
- When a match is found, the statement sequence associated with that case executes until a break statement or the end of the switch is reached
- The default statement executes if no matches are found — it is optional; if not present, no action takes place when all matches fail
- C89 specifies that a switch can have at least 257 case statements; C99 requires at least 1,023

Three important rules about switch:

Rule	Detail
Equality only	switch can only test for equality; it can evaluate any relational or logical expression
Unique constants	No two case constants in the same switch can have identical values (inner switches may share constants with outer)
Char auto-conversion	If character constants are used, they are automatically converted to integers per C's type conversion rules

Example — menu selection with switch:

```
void menu(void)
{
    char ch;
    printf("1. Check Spelling\n");
    printf("2. Correct Spelling Errors\n");
    printf("3. Display Spelling Errors\n");
    printf("Enter your choice: ");
    ch = getchar();
    switch(ch) {
        case '1': check_spelling(); break;
        case '2': correct_errors(); break;
        case '3': display_errors(); break;
        default: printf("No option
selected");
    }
}
```

```
}
```

break and Fall-Through in switch

Technically, the break statements inside switch are optional. They terminate the statement sequence associated with each constant.

- If break is omitted, execution falls through into the next case's statements until a break or end of switch is reached
- Fall-through can be used to prevent unnecessary duplication of statements, resulting in more efficient code

Example — fall-through for grouping cases:

```
void inp_handler(int i)
{
    int flag = -1;
    switch(i) {
        case 1: /* These cases share */
        case 2: /* the same statement
sequence */
        case 3:
            flag = 0;
            break;
        case 4:
            flag = 1; /* falls through to
case 5 — no break! */
        case 5:
            error(flag);
            break;
        default:
            process(i);
    }
```

```
    }  
}
```

NOTE *If i matches 4, flag is set to 1 and because there is no break, execution continues into case 5, calling `error(flag)`. If i matches 5, `error(flag)` is called with flag value of -1 (its initial value).*

Nested switch Statements

You can have a switch as part of the statement sequence of an outer switch. Even if the case constants of the inner and outer switch share common values, no conflicts arise.

```
switch(x) {  
    case 1:  
        switch(y) { /* nested switch */  
            case 0: printf("Divide by zero  
error.\n"); break;  
            case 1: process(x, y); break;  
        }  
        break;  
    case 2:  
        .  
        .  
}
```

ITERATION STATEMENTS

In C, iteration statements (also called loops) allow a set of instructions to be repeatedly executed until a certain condition is reached.

Loop	Condition Check	Minimum Executions	Best Used When
for	Top (before body)	0	Number of iterations is known or uses a loop control variable
while	Top (before body)	0	Condition may be false initially; loop body may not execute at all
do-while	Bottom (after body)	1 (always)	Loop body must execute at least once (e.g., menu selection)

The for Loop

The general form of the for statement is:

```
for (initialization; condition; increment)
    statement;
```

Section	Purpose
initialization	An assignment statement that sets the loop control variable

Section	Purpose
condition	A relational expression that determines when the loop exits; the loop continues as long as this is true
increment	Defines how the loop control variable changes each time the loop repeats

- The three sections are separated by semicolons
- The conditional test is always performed at the top of the loop — body may not execute at all if condition is initially false
- Once condition becomes false, execution resumes on the statement immediately following the for

Example — print 1 through 100:

```
#include <stdio.h>
int main(void)
{
    int x;
    for(x=1; x <= 100; x++) printf("%d ",
x);
    return 0;
}
```

for Loop Variations

The for loop is very flexible. Several important variations exist:

1. Multiple loop control variables using the comma operator:

```
for(x=0, y=0; x+y < 10; ++x) {  
    y = getchar();  
    y = y - '0';  
}
```

NOTE *Commas separate the initialization statements. Both `x` and `y` must be at the correct value for the loop to terminate. `y` must be initialized to 0 so its value is defined before the first conditional evaluation.*

Example — converge() function demonstrating multiple control variables:

```
/* Copies a string by moving characters  
from both ends toward the middle */  
void converge(char *targ, char *src)  
{  
    int i, j;  
    printf("%s\n", targ);  
    for(i=0, j=strlen(src); i<=j; i++, j--)  
    {  
        targ[i] = src[i];  
        targ[j] = src[j];  
        printf("%s\n", targ);  
    }  
}
```

2. Complex conditional expressions — the condition does not have to test the loop control variable:

```
/* Loop terminates when 3 tries are used OR
correct password is entered */
for(x=0; x<3 && strcmp(str, "password");
++x) {
    printf("Enter password please: ");
    gets(str);
}
```

3. Function calls in for sections — each section can be any valid expression:

```
for(prompt(); t=readnum(); prompt())
    sqrnum(t);
/* prompt() initializes, readnum() is the
condition and also reads input,
prompt() increments by displaying the
prompt again. Loop ends when 0 is entered.
*/
```

4. Missing sections — any or all of the three expressions are optional:

```
for(x=0; x != 123; )    /* no increment */
    scanf("%d", &x);

/* Initialization outside for, increment
inside body */
gets(s);
if(*s) x = strlen(s);
else x = 10;
for( ; x < 10; ) {
```

```
    printf("%d", x);  
    ++x;  
}
```

The Infinite Loop

The for loop is traditionally used to create infinite loops. Leaving the conditional expression empty creates an endless loop:

```
for( ; ; ) printf("This loop will run  
forever.\n");
```

- When the conditional expression is absent, it is assumed to be true
- `for(;;)` is C programmers' conventional way to signify an infinite loop
- A `break` statement anywhere inside the loop causes immediate termination — so it is not truly infinite if `break` is used

```
ch = '\0';  
for( ; ; ) {  
    ch = getchar();  
    if(ch == 'A') break; /* exit the loop  
*/  
}  
printf("you typed an A");
```

for Loops with No Body (Empty Loops)

A statement may be empty, which means the body of a for loop may also be empty. This is useful to simplify coding of certain algorithms and to create time delay loops.

```
/* Advance past leading spaces in string
pointed to by str */
for( ; *str == ' '; str++) ;    /* no body
needed */

/* Time delay loop */
for(t=0; t < SOME_VALUE; t++) ;
```

NOTE *Some compilers may optimize a time delay loop out of existence since it has no effect as far as the compiler is concerned. The expected delay may not always be produced.*

Declaring Variables within a for Loop

In C99 (and C++), but NOT in C89, it is possible to declare a variable within the initialization portion of a for loop. Such a variable has its scope limited to the block of code controlled by that loop.

```
/* Valid in C99/C++, INVALID in C89 */
int j;
for(int i = 0; i < 10; i++)
    j = i * i;
/* i = 10; *** Error *** -- i is not known
outside the loop! */
```

- Declaring a loop control variable in the initialization portion is becoming common practice
- Remember: this is not supported by C89

The while Loop

The general form of the while loop is:

```
while(condition) statement;
```

- Statement can be an empty statement, a single statement, or a block of statements
- The condition may be any expression; true is any nonzero value
- The loop iterates while the condition is true
- Like the for loop, the condition is checked at the TOP — body may not execute if condition is initially false
- When condition becomes false, execution passes to the line immediately following the loop

Example — keyboard input routine that loops until user types A:

```
char wait_for_char(void)
{
    char ch;
    ch = '\0'; /* initialize ch */
    while(ch != 'A') ch = getchar();
    return ch;
}
```

Example — pad() function adds spaces to a string; the while condition prevents execution if string is already at the desired length:

```
void pad(char *s, int length)
{
    int l;
    l = strlen(s);
    while(l < length) {
        s[l] = ' ';
        l++;
    }
    s[l] = '\\0';
}
```

A common pattern is a single loop-control variable that can be set to false from multiple places within the loop:

```
void func1(void)
{
    int working = 1;  /* true */
    while(working) {
        working = process1();
        if(working) working = process2();
        if(working) working = process3();
    }
}
/* Any of the three routines may return
false and cause the loop to exit */
```

The body of a while loop may also be empty:

```
while((ch=getchar()) != 'A') ; /* loops
until user types A */
```

The do-while Loop

Unlike for and while loops, the do-while loop checks its condition at the **BOTTOM** of the loop. This means a do-while loop always executes at least once.

General form:

```
do {
    statement;
} while(condition);
```

- The curly braces are not strictly necessary when only one statement is present, but they are usually used to avoid confusion with while
- The loop iterates until condition becomes false
- Most common use: menu selection functions — the menu must display and process at least once

Example — reads numbers until one ≤ 100 is entered:

```
do {
    scanf("%d", &num);
} while(num > 100);
```

Example — menu function using do-while:

```
void menu(void)
{
    char ch;
    printf("1. Check Spelling\n");
    printf("2. Correct Spelling Errors\n");
    printf("3. Display Spelling Errors\n");
    printf("Enter your choice: ");
    do {
        ch = getchar();
        switch(ch) {
            case '1': check_spelling();
break;
            case '2': correct_errors();
break;
            case '3': display_errors();
break;
        }
    } while(ch!='1' && ch!='2' && ch!='3');
}
```

NOTE *The do-while loop is a good choice for menu functions because you always want them to execute at least once. The loop continues until a valid option is selected.*

JUMP STATEMENTS

C has four statements that perform an unconditional branch:

Jump Statement	Where It Can Be Used
return	Anywhere inside a function
goto	Anywhere inside a function (but only to labels in the same function)
break	Inside any loop (for, while, do-while) or inside a switch statement
continue	Inside any loop (for, while, do-while)

The return Statement

The return statement is used to return from a function. It is a jump statement because it causes execution to jump back to the point at which the call to the function was made.

General form:

```
return expression;
```

- A return may or may not have a value associated with it
- A return WITH a value can be used only in a function with a non-void return type — the value becomes the return value of the function
- A return WITHOUT a value is used to return from a void function
- You can use as many return statements as you like within a function — the function stops executing as soon as it encounters the first return
- The } that ends a function also causes the function to return (same as return without a value)

- A function declared as void cannot contain a return statement that specifies a value

Standard	Rule for return
C89	A return statement in a non-void function does not HAVE to return a value; if no value is specified, a garbage value is returned
C99	A return statement in a non-void function MUST return a value
C++ (same as C99)	A return statement in a non-void function MUST return a value

NOTE *Even for C89, if a function is declared as returning a value, it is good practice to actually return one!*

The goto Statement

Since C has a rich set of control structures along with break and continue, there is little need for goto. Its chief concern is its tendency to render programs unreadable (spaghetti code). Nevertheless, it occasionally has uses — for example, jumping out of a set of deeply nested loops.

The goto statement requires a label. A label is a valid identifier followed by a colon. The label must be in the same function as the goto.

General form:

```
goto label;
.
.
.
label:    /* label must be in the same
function */
```

Example — creating a loop from 1 to 100 using goto:

```
x = 1;
loop1:
x++;
if(x <= 100) goto loop1;
```

NOTE *The goto is not commonly used and is discouraged in structured programming. There are no programming situations that absolutely require goto — it is only a convenience for a narrow set of situations.*

The break Statement

The break statement has two uses:

- Terminate a case in a switch statement
- Force immediate termination of a loop, bypassing the normal loop conditional test

When break is encountered inside a loop, the loop is immediately terminated and program control resumes at the next statement following the loop.

```
#include <stdio.h>
int main(void)
{
    int t;
    for(t=0; t < 100; t++) {
        printf("%d ", t);
        if(t == 10) break; /* exits when t
reaches 10 */
    }
    return 0;
}
/* Output: 0 1 2 3 4 5 6 7 8 9 10 */
```

- **break** causes an exit from **ONLY** the innermost loop — an outer loop continues normally
- A **break** used in a switch will affect only that switch — it does not affect any loop the switch happens to be in

Example — breaking out of inner loop only (outer loop continues):

```
for(t=0; t < 100; ++t) {
    count = 1;
    for(;;) {
        printf("%d ", count);
        count++;
        if(count == 10) break; /* only
exits inner for(;;) */
    }
}
```

```
/* Prints 1 through 9 on screen 100 times
*/
```

Example — using break for special termination condition:

```
void look_up(char *name)
{
    do {
        /* look up names ... */
        if(kbhit()) break; /* exit if key
is pressed */
    } while(!found);
    /* process match */
}
```

The exit() Function

Although `exit()` is not a program control statement (it is a library function), it is related to `break`. `exit()` causes immediate termination of the entire program, forcing a return to the operating system — as if it were breaking out of the entire program.

General form:

```
void exit(int return_code);
```

- The value of `return_code` is returned to the calling process (usually the OS)
- Zero (0) is commonly used to indicate normal program termination

- Non-zero values indicate some sort of error
- The macros `EXIT_SUCCESS` and `EXIT_FAILURE` can also be used as `return_code`
- Requires the header `<stdlib.h>`

Example — exiting if required hardware is absent:

```
#include <stdlib.h>
int main(void)
{
    if(!virtual_graphics()) exit(1);    /*
terminate if no graphics adapter */
    play();
}
```

The continue Statement

The continue statement works somewhat like break. Instead of forcing loop termination, it forces the NEXT iteration of the loop to take place, skipping any code in between.

Loop Type	Effect of continue
for	Causes the increment expression to execute, then the conditional test
while	Program control passes directly to the conditional test
do-while	Program control passes directly to the conditional test

Example — count spaces in a string using continue:

```
/* Count spaces */
#include <stdio.h>
int main(void)
{
    char s[80], *str;
    int space;
    printf("Enter a string: ");
    gets(s);
    str = s;
    for(space=0; *str; str++) {
        if(*str != ' ') continue; /* skip
non-space characters */
        space++;
    }
    printf("%d spaces\n", space);
    return 0;
}
```

Example — using continue to force early exit from a loop:

```
/* Shift-cipher: shift all characters one
letter higher; stop at $ */
void code(void)
{
    char done, ch;
    done = 0;
    while(!done) {
        ch = getchar();
        if(ch == '$') {
            done = 1;
        }
    }
}
```

```

        continue; /* jump to
while(!done) test immediately */
    }
    putchar(ch+1);
}
/* After $ is input, continue causes the
conditional test to execute,
which finds done=1 and exits the loop.
No further output occurs. */

```

EXPRESSION STATEMENTS

An expression statement is simply a valid expression followed by a semicolon. Remember, Chapter 2 covers expressions thoroughly — this section highlights special points.

```

func();          /* a function call —
expression statement */
a = b + c;       /* an assignment statement
*/
b + f();         /* a valid, but unusual
statement — f() may perform a needed task
*/
;               /* an empty statement (also
called a null statement) */

```

Example	Description
<code>func();</code>	Executes a function call; the function's side effect is the purpose

Example	Description
<code>a = b+c;</code>	An assignment — evaluates the expression and stores the result
<code>b+f();</code>	Strange but valid — compiled because <code>f()</code> may perform a necessary task even though the result of <code>b+f()</code> is discarded
<code>;</code>	Empty statement (null statement) — a statement that does nothing; valid anywhere a statement is expected

NOTE *The third form (`b+f();`) is valid because the compiler evaluates the function `f()` for its possible side effects. The result of the expression is simply discarded.*

BLOCK STATEMENTS

Block statements are groups of related statements that are treated as a unit. They are also called compound statements.

- A block is begun with `{` and terminated by its matching `}`
- The statements that make up a block are logically bound together
- Most commonly used to create a multi-statement target for another statement, such as `if`
- A block statement may be placed anywhere you would put any other statement — including free-standing blocks

Example — free-standing block statement (unusual but valid):

```
#include <stdio.h>
int main(void)
{
    int i;
    { /* a free-standing block statement
*/
        i = 120;
        printf("%d", i);
    }
    return 0;
}
```

Feature	Block Statement Behavior
Delimiters	Opened with { and closed with }
Local variables	Variables declared inside a block are local to that block and destroyed when the block ends
Common usage	Multi-statement targets for if, else, for, while, do-while
Alternative name	Compound statement
Free-standing	A block can appear anywhere in a program that a single statement is valid

CHAPTER 3 — QUICK REFERENCE SUMMARY

Statement	Category	Key Points
if / else	Selection	Executes a statement or block based on a condition; else is optional; only one branch executes
if-else-if ladder	Selection	Tests a series of conditions top-down; first true condition executes; final else is a catch-all
? (ternary)	Selection	Compact alternative to if-else; evaluates to Exp2 if Exp1 is true, else Exp3
switch	Selection	Tests a single integer/char expression against case constants; break exits; default handles no-match
for	Iteration	Uses init; condition; increment; condition tested at top; powerful variations possible
while	Iteration	Condition tested at top; body may not execute if condition initially false
do-while	Iteration	Condition tested at bottom; body ALWAYS executes at least once
return	Jump	Returns from a function; may return a value; multiple returns allowed per function

Statement	Category	Key Points
goto	Jump	Unconditional jump to a label; label must be in same function; seldom recommended
break	Jump	Exits the innermost loop or switch immediately; does not affect outer loops
continue	Jump	Skips rest of loop body; forces next iteration; for: runs increment first
exit()	Library	Terminates entire program; returns code to OS; requires <stdlib.h>
Expression	Expression	Any valid expression + semicolon; includes function calls, assignments, null statement (;)
Block / Compound	Block	Group of statements between { }; treated as a unit; variables declared inside are local

— End of Chapter 3 Notes —